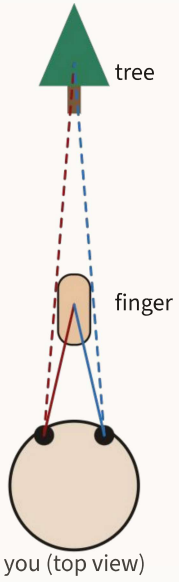
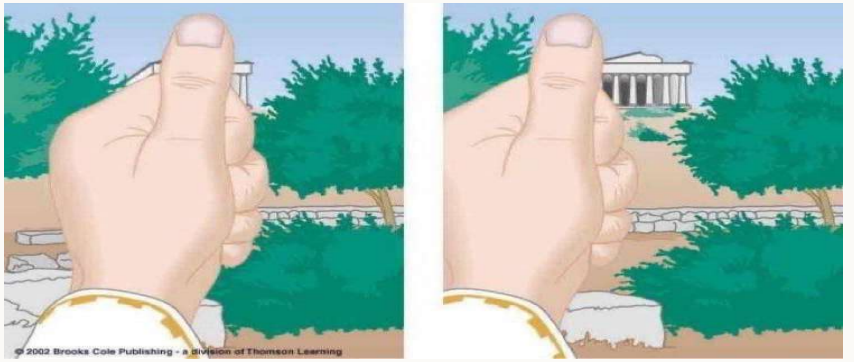
Two eyes: the original stereo system



What the left eye sees

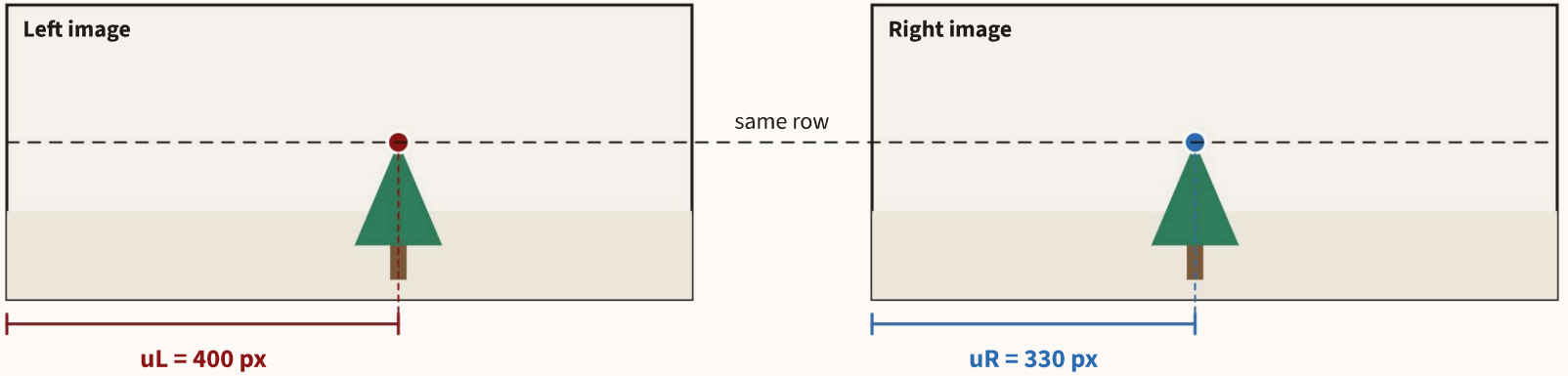
What the right eye sees



finger jumps a lot · tree barely moves

Try it: hold up a finger, close one eye, then the other. That jump is **disparity**.

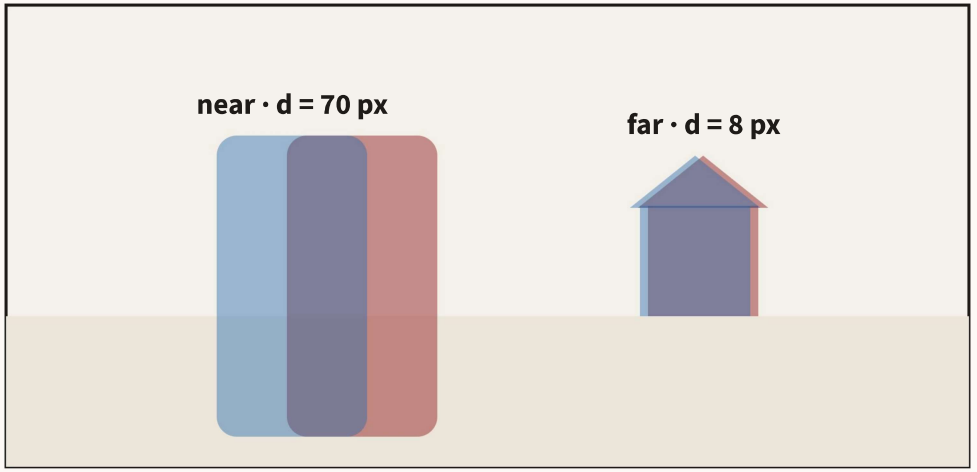
Disparity: the horizontal shift



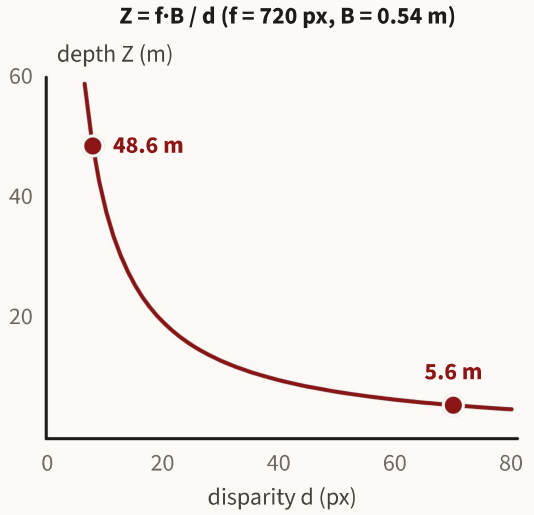
$$d = u\mathrm{L} - u\mathrm{R} = 400 - 330 = \mathbf{70\ px}$$

After rectification the match is on the **same row**. The only unknown left is a horizontal shift: the **disparity**.

Near objects shift more



red = left image · blue = right image, overlaid

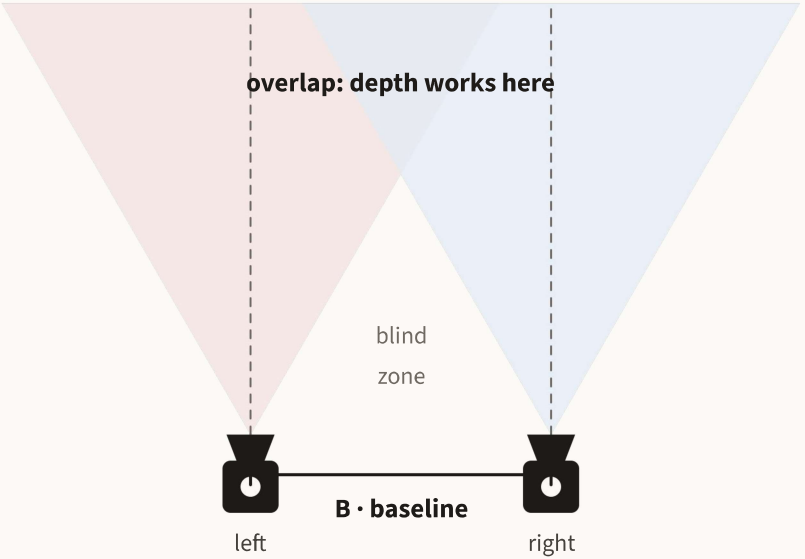


Stereo cameras and the baseline

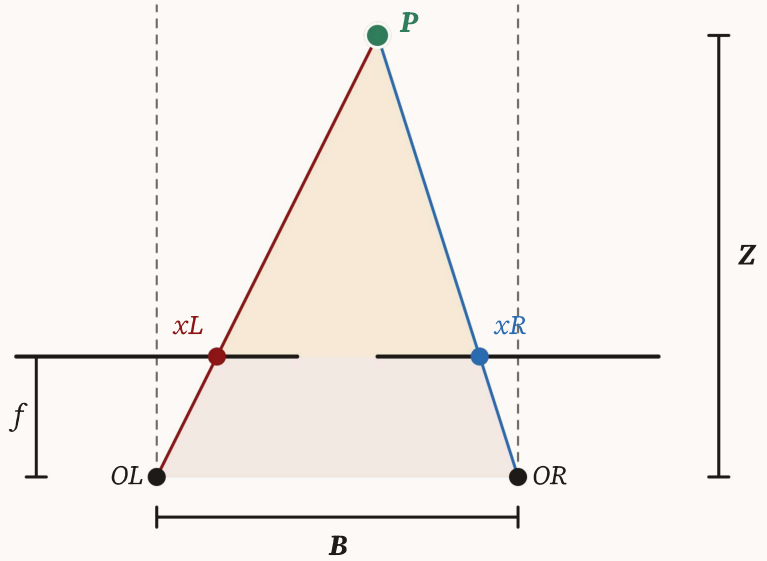
Two cameras, same orientation, side by side.

- **Baseline B** = distance between the optical centers
- Human eyes: B ≈ 6.5 cm
- A car stereo rig: tens of centimeters (KITTI: ≈ 54 cm)

> **Trade-off**
> Wider B → better depth far away, but a bigger blind zone and harder matching.



The depth equation



$$Z = f \cdot B / d$$

Triangle *P–OL–OR* ~ triangle *P–xL–xR* (similarity rule)

$B / Z = (B - d) / (Z - f) \Rightarrow Z = fB / d$

Worked example

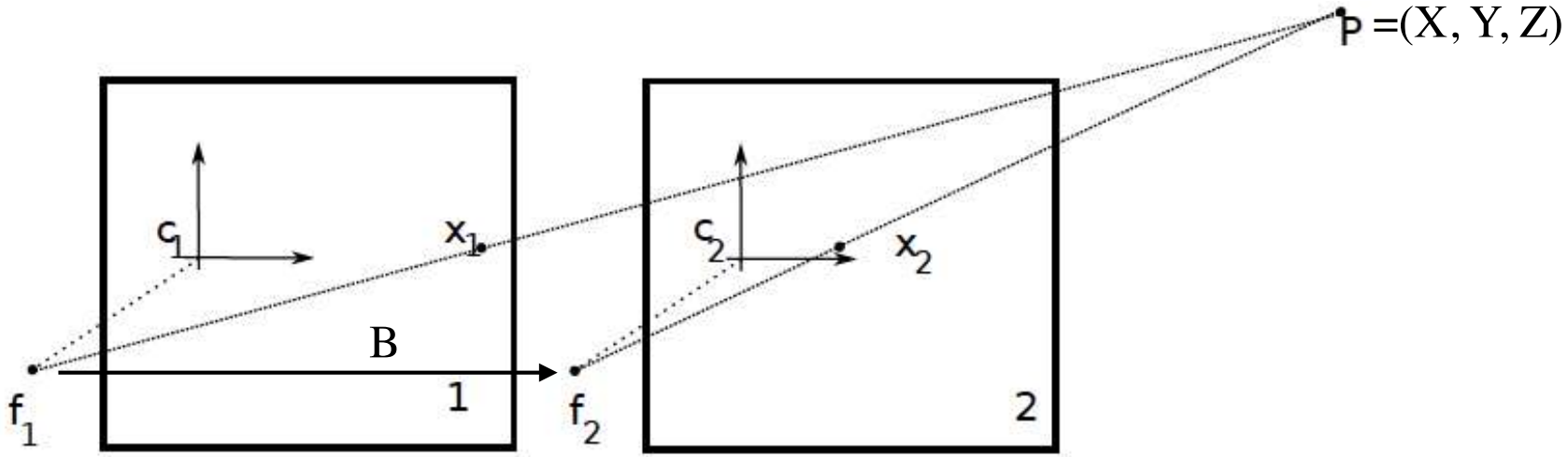
f = 720 px, B = 0.54 m, d = 24 px → Z = 720 × 0.54 / 24 = **16.2 m**

Off by 1 px (d = 23) → 16.9 m. Error grows like Z^2.



Disparity

Assume camera 2 is translated wrt camera 1, both are calibrated



Disparity

$$d = x_2 - x_1 = f\frac{(X - B) - X}{Z} = -f\frac{B}{Z}$$

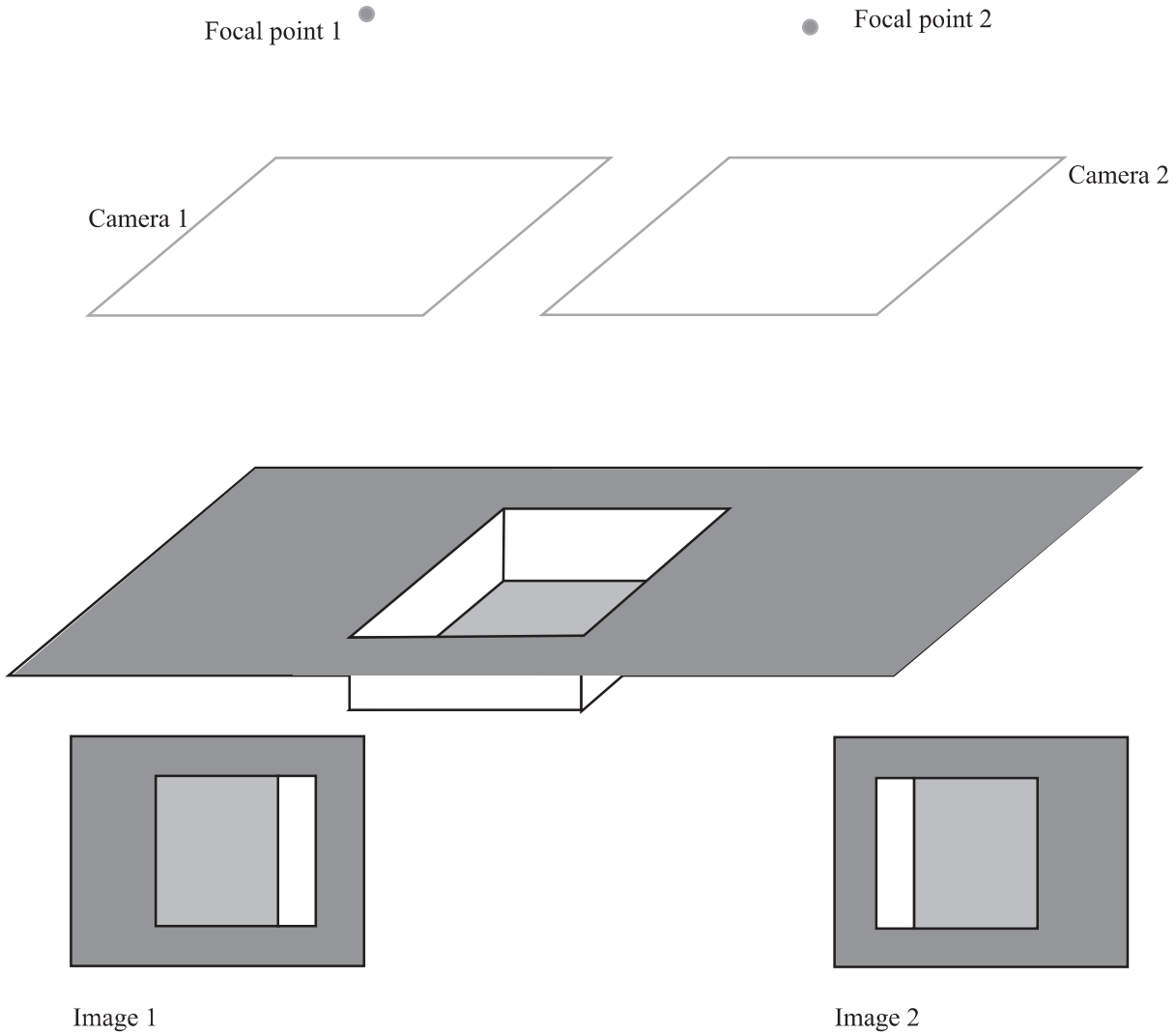
Baseline

Depth

Pragmatics

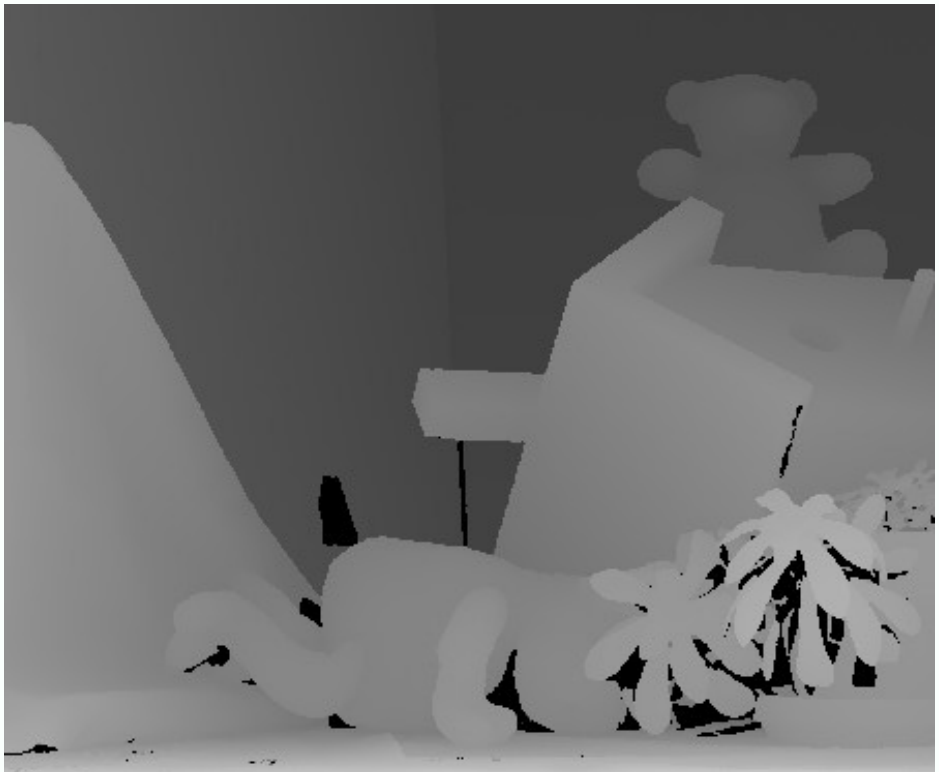
- Issue
  - Match points
- Strategy
  - correspondences occur only along relevant epipolar line
  - represent points with local representation
    - color + surface smoothness; filters; some kind of deep feature
- Issue
  - some points don’t have correspondences (occlusion)
- Match left to right, then right to left
  - if they don’t agree, break match

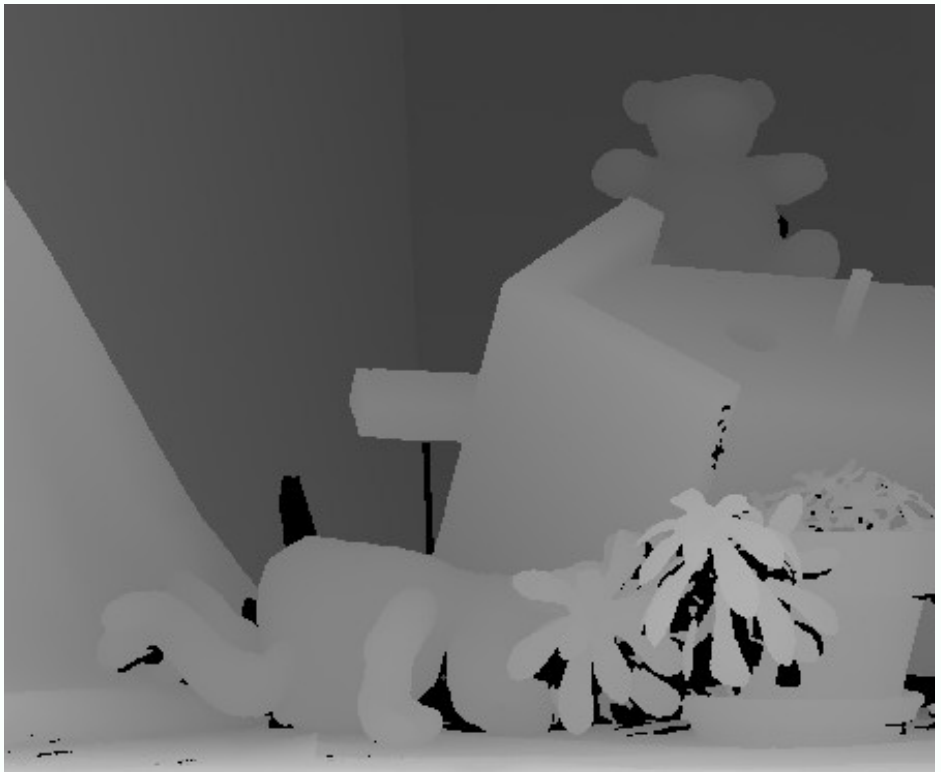
Some points don’t have matches



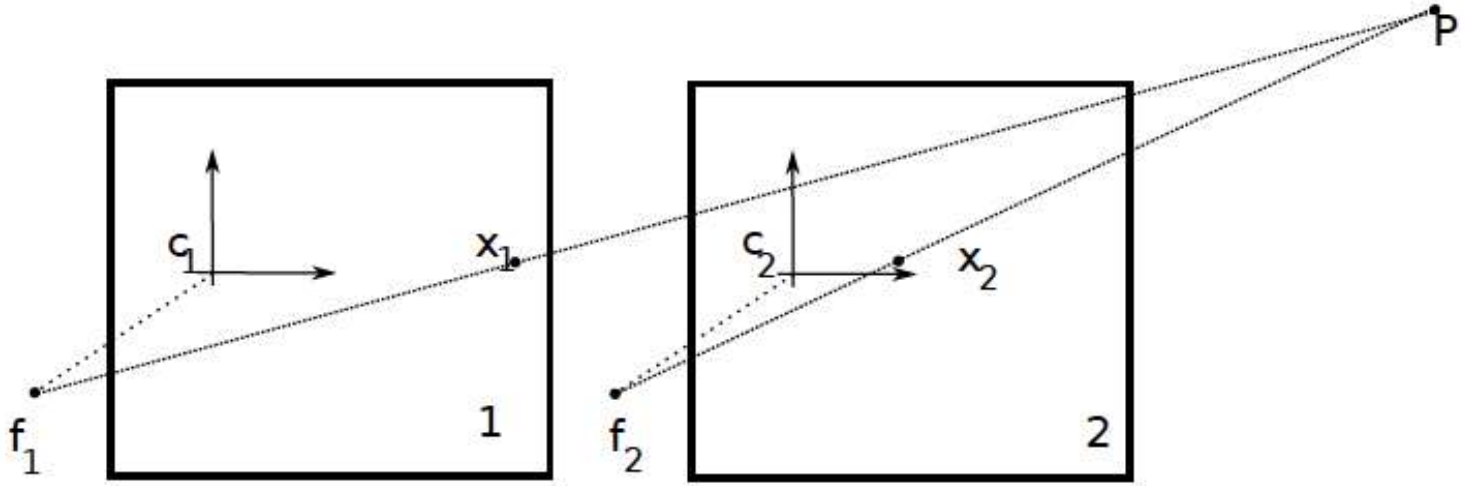








Stereo as an optimization problem

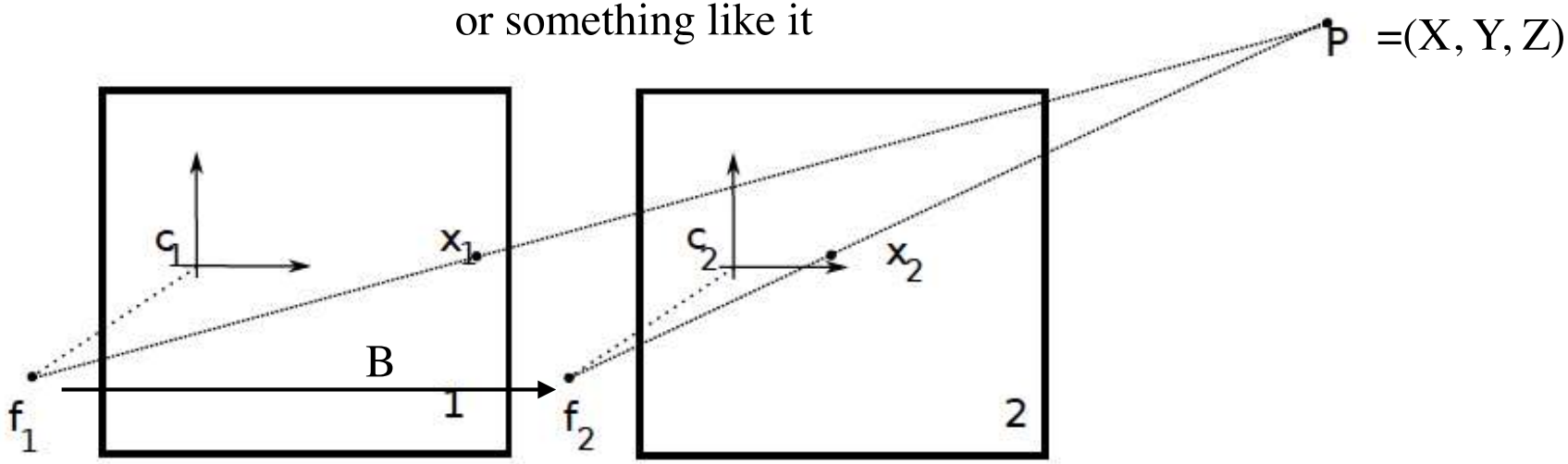


- Original:
  - find q, q' that match, and infer depth
- Now:
  - choose value of depth at q; then quality of match at q' is cost
  - optimize this

Stereo as an optimization problem

Assume camera 2 is translated wrt camera 1, both are calibrated

IF you choose the right depth for x_1, then:
you know disparity, so you know x_2
and you can optimize
||color(x_1)-color(x_2(d))||^2
or something like it



Disparity

$$d = x_2 - x_1 = f\frac{(X - B) - X}{Z} = -f\frac{B}{Z}$$

Baseline

Depth

Stereo as an optimization problem

- Typically:
  - quantize depth to a fixed number of levels
  - two costs:
    - compare x_1, x_2(d) — unary cost
      - color match (photometric consistency constraint)
      - it can be helpful to match intensity gradients, too
    - compare x_1, x_1'
      - typically, smoothness constraint on recovered depths
      - eg depth gradient not too big, etc.
  - massive discrete quadratic program

Stereo as an optimization problem (II)

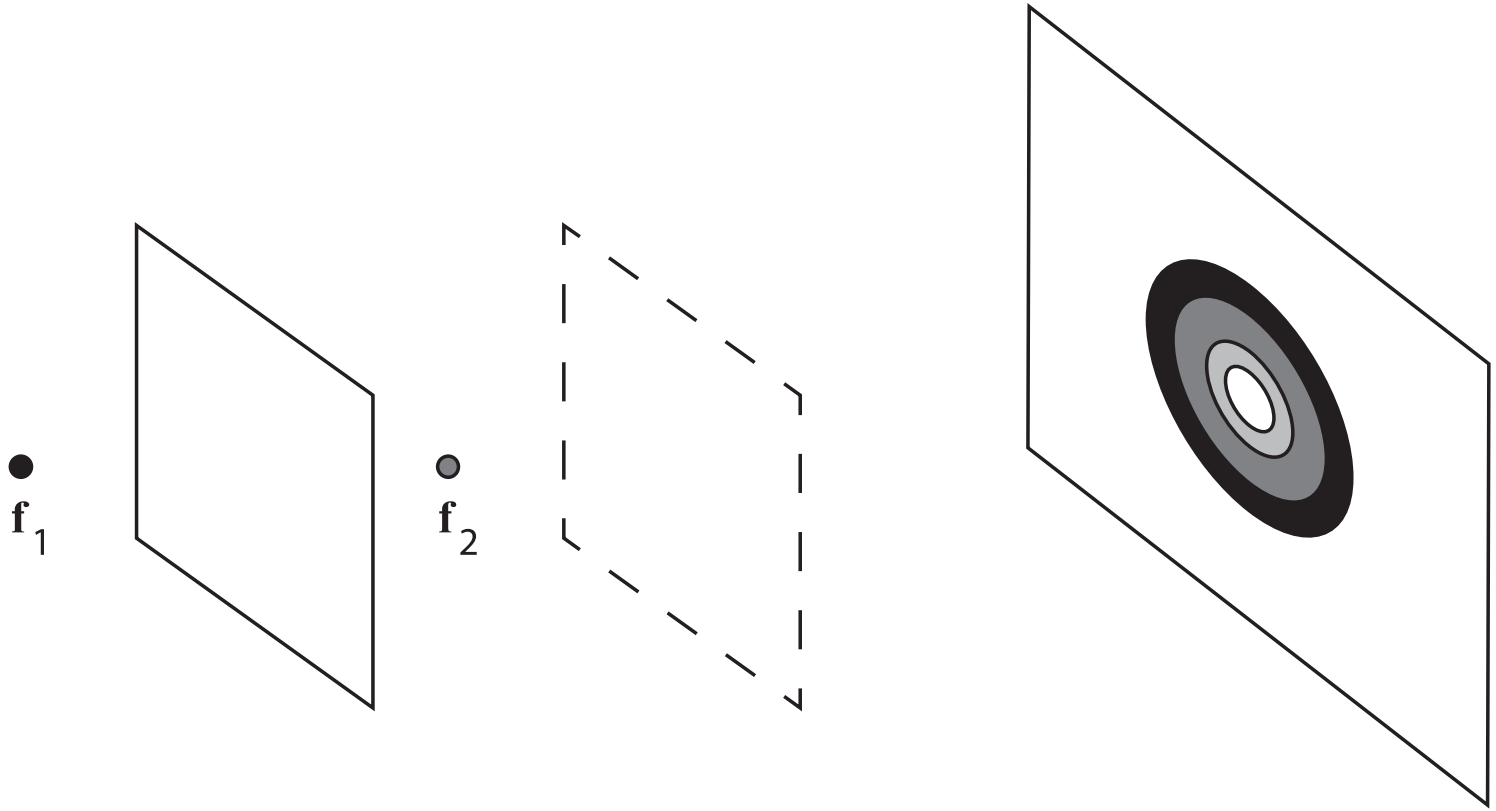
- Segment images into regions
  - NOT semantic; small, constant color+texture
- Each region is assumed to have a linear disparity
  - $d(x, y)=a\ x + b\ y+c$
- Find a quantized "vocabulary" of such disparities
  - eg by initial disparity, incremental fitting
- For each region, choose the "best" in the "vocabulary"
  - This is a discrete optimization problem
  - It's quadratic
    - unary term - does the chosen vocab item "agree" with color data?
    - binary term - are neighboring pairs of models "similar" on boundary?

Stereo resources

- Datasets and evaluations:
  - Middlebury stereo page has longstanding
    - datasets
    - evaluations with leaderboards
    - datasets with groundtruth
    - refs to other such collections
      - (but this is the best known, by a long way)
  - https://vision.middlebury.edu/stereo/

Optic flow

- Generically:
  - a “small” camera movement yields image 2 from image 1
  - determine where points in image 1 move
- Assume we’re moving rigidly in a stationary environment
  - then points will move along their epipolar lines
    - where the epipolar lines follow from fundamental matrix
      - so from camera movement
- Main point of contrast with stereo
  - Images are not usually simultaneous
    - so objects might have moved



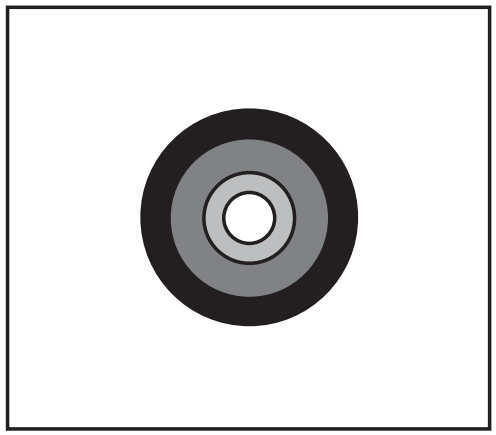


Image 1

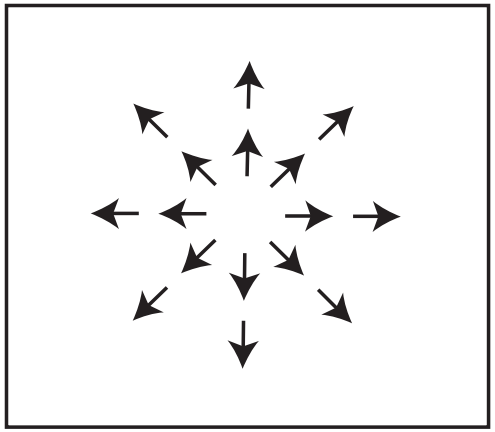


Image 1 optic flow

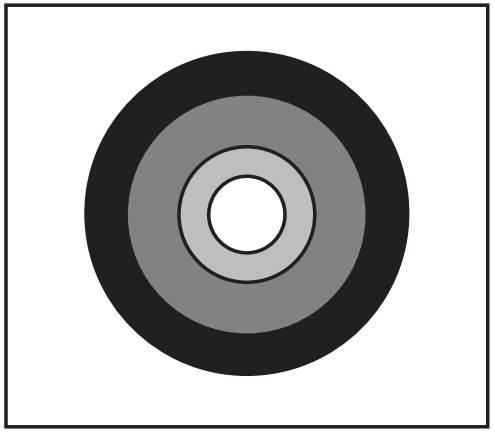
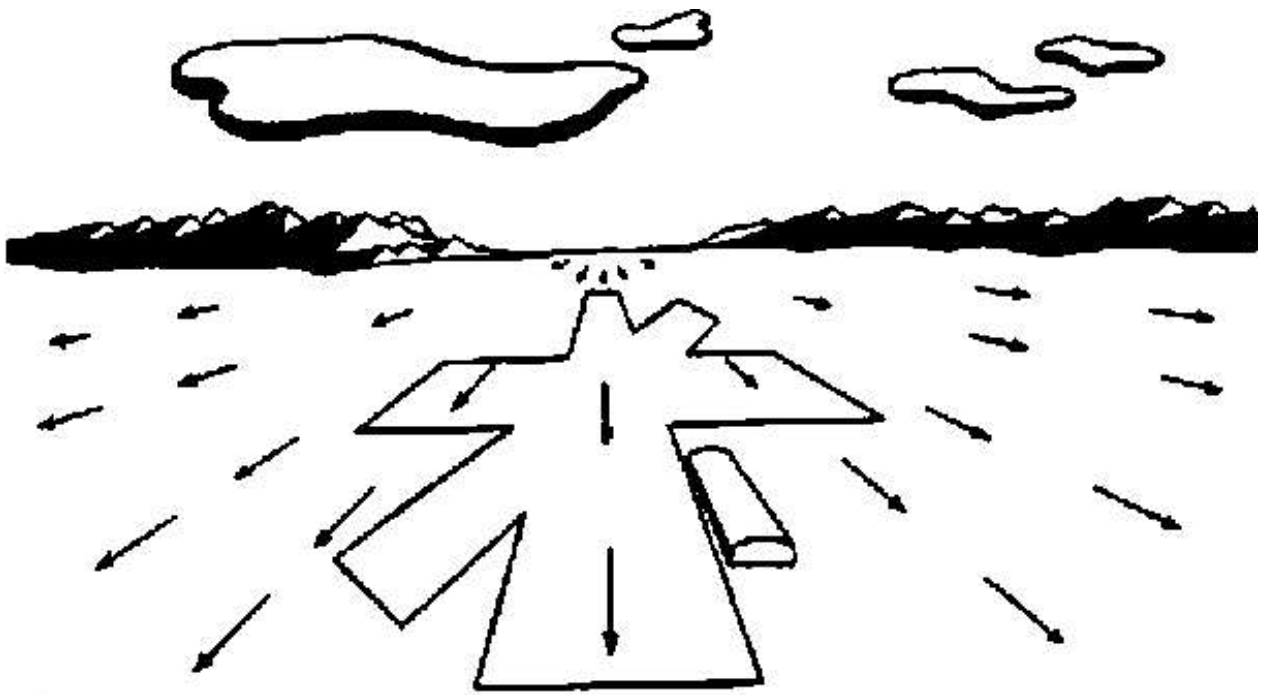
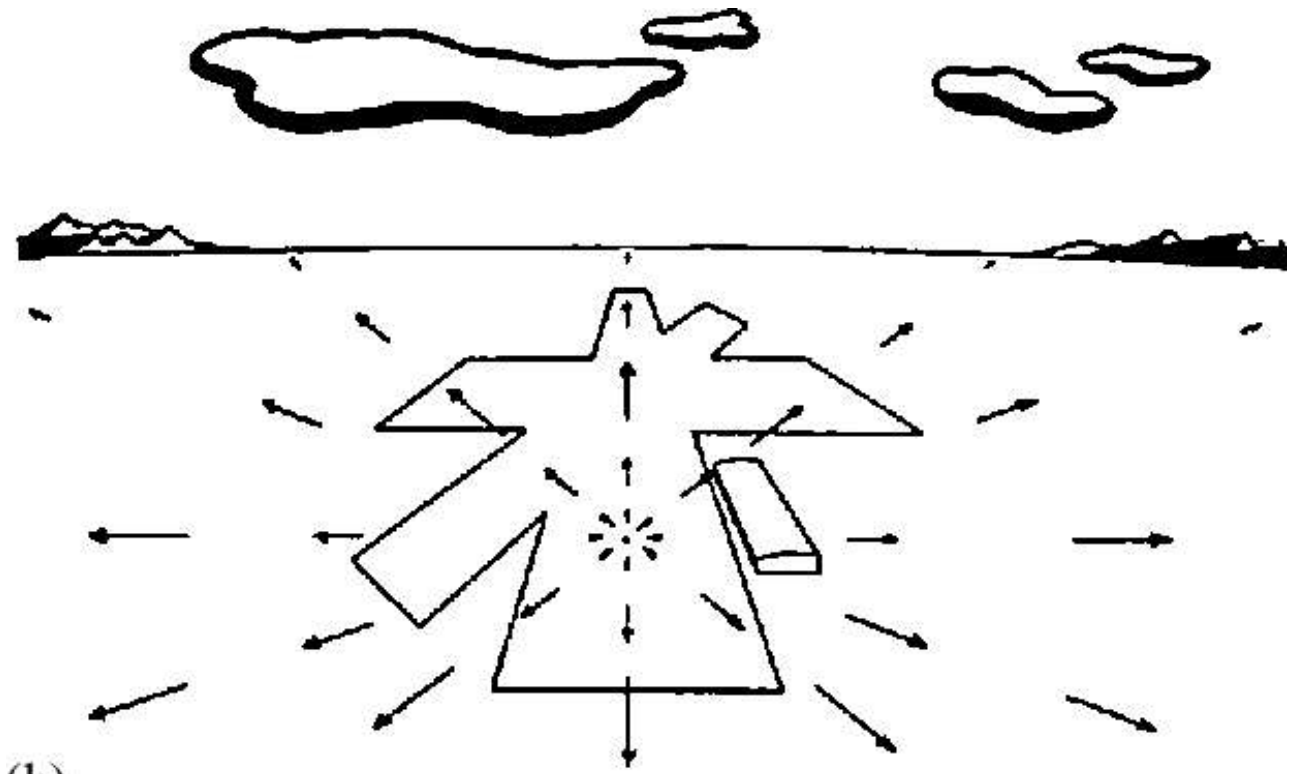


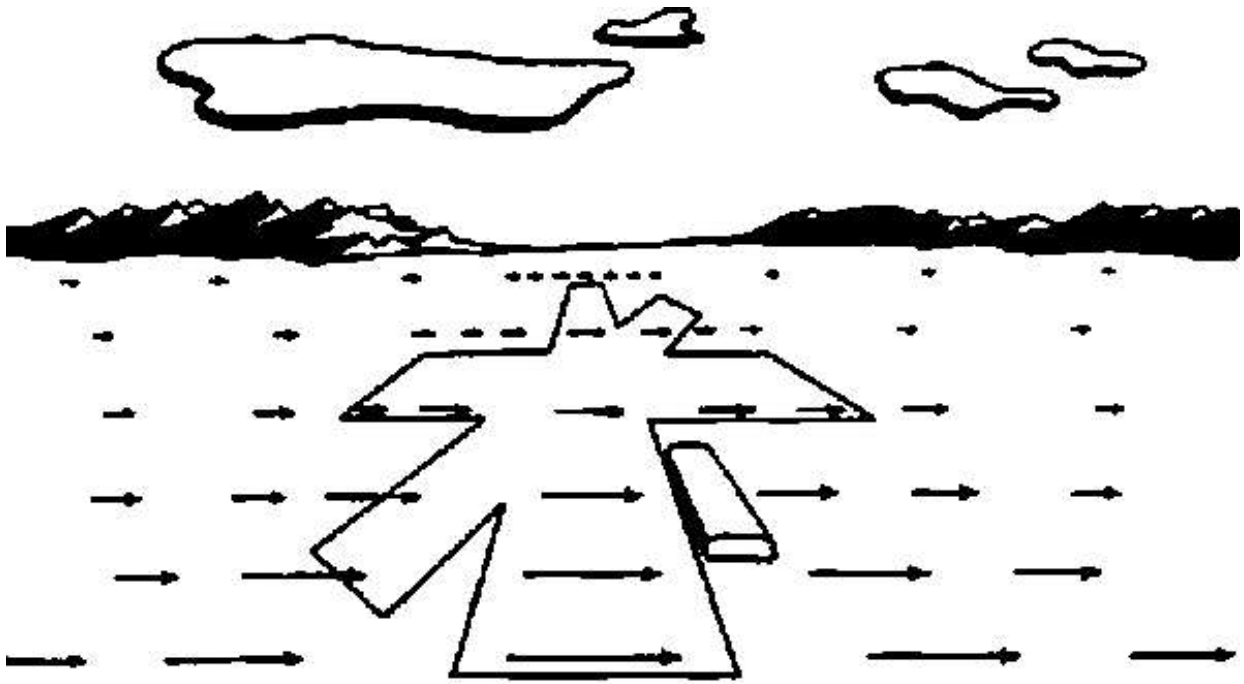
Image 2



(a)

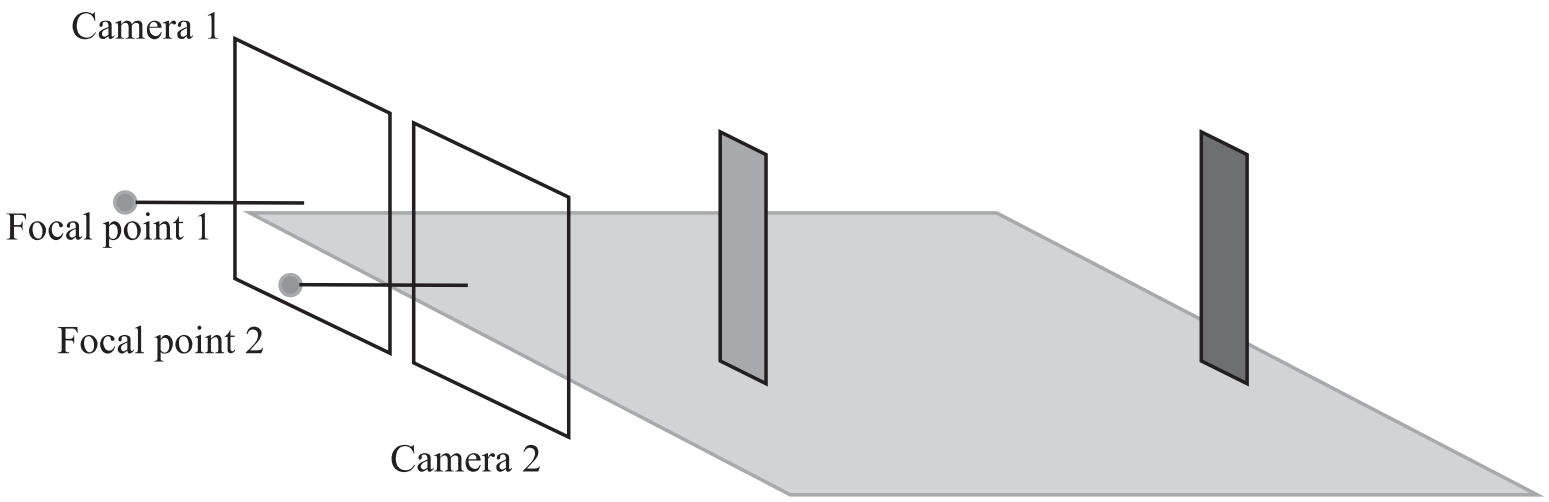


(b)



Optical flow

- Generically:
  - a “small” camera movement yields image 2 from image 1
  - determine where points in image 1 move
- Assume we’re moving rigidly in a stationary environment
  - then points will move along their epipolar lines
    - where the epipolar lines follow from fundamental matrix
      - so from camera movement
- As we saw, HOW FAR they move is determined by depth
  - and by their movement!!!



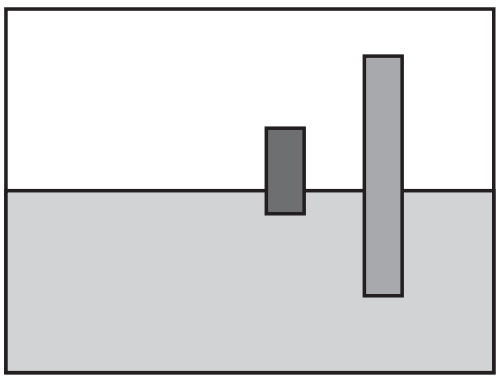


Image 1

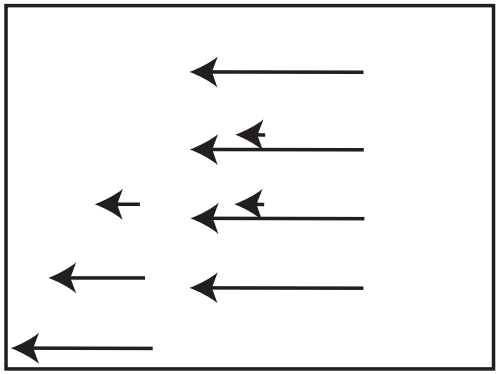


Image 1

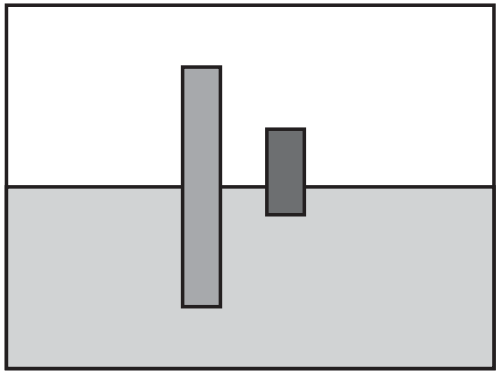


Image 2

Strategy

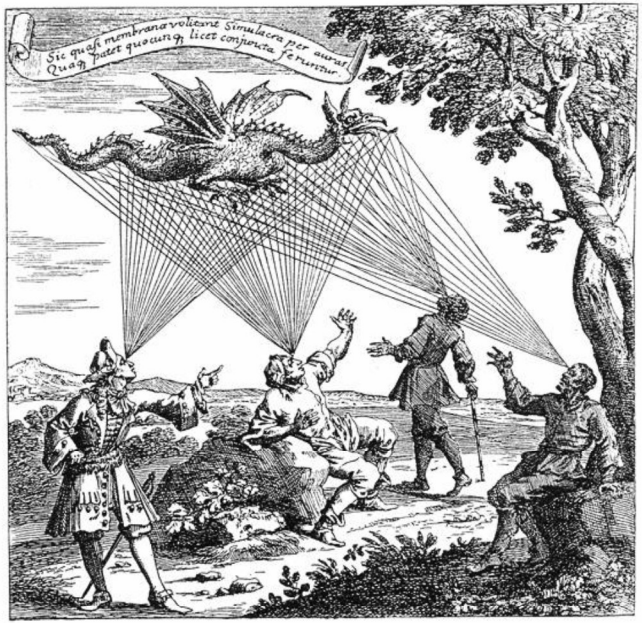
- Segment into regions, estimate region correspondences
  - use to inform flow estimate



Figure 9. **Left:** Two overlaid images of a tennis player in action. **Center left:** Region correspondences. **Center right:** Result with optical flow from [4]. The motion of the right leg is too fast to be estimated. **Right:** The proposed method captures the motion of the leg.

Brox et al 09

Structure from motion

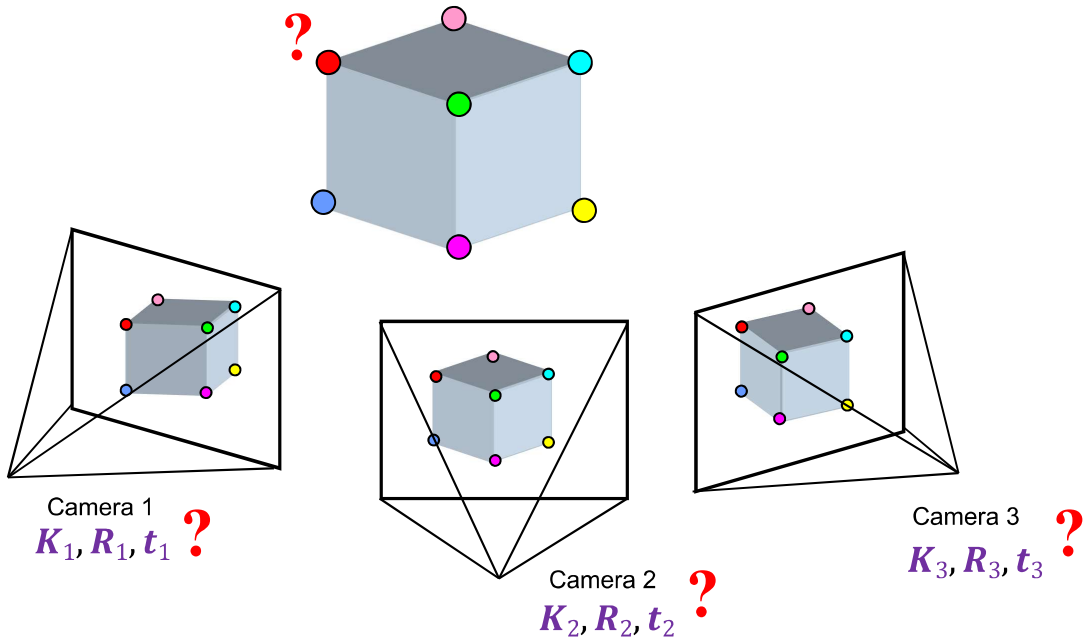


Драконъ, видимый подъ различными углами зрѣнія
По гравюрѣ на мѣди изъ „Oculus artificialis teledioptricus“ Цана. 1702 года.

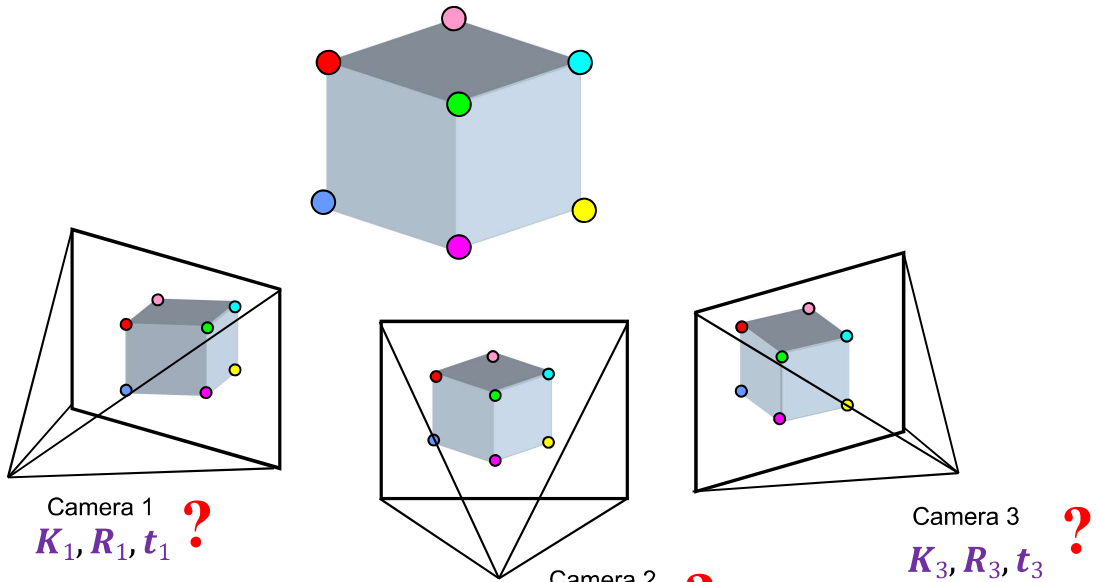
Outline: Structure from motion

- Problem definition and ambiguities
- Affine structure from motion
  - Factorization
- Projective structure from motion
  - Bundle adjustment
- Modern structure from motion pipeline

Structure from motion

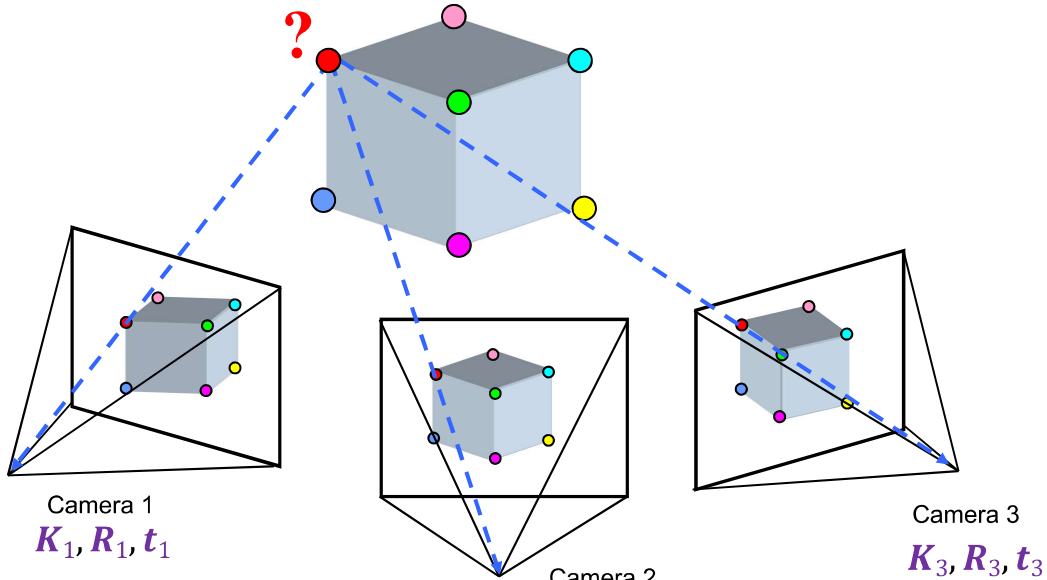


Recall: Calibration



- Given a set of *known* 3D points seen by a camera, compute the camera parameters

Recall: Triangulation



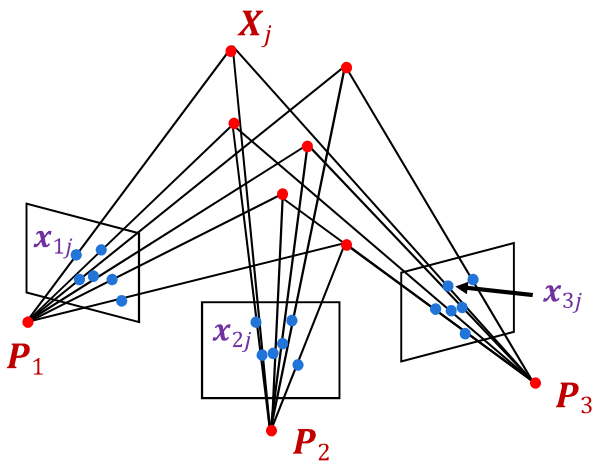
- Given *known cameras* and projections of the same 3D point in two or more images, compute the 3D coordinates of that point

Structure from motion: Problem formulation

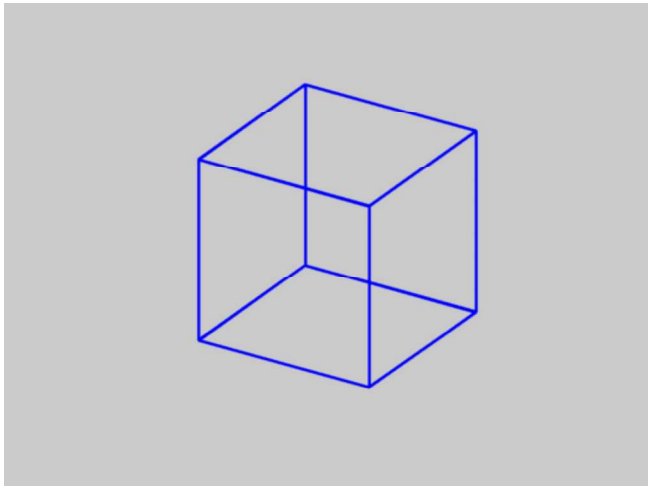
- Given: m images of n fixed 3D points such that (ignoring visibility)

$$\boldsymbol{x}_{ij} \cong \boldsymbol{P}_i \boldsymbol{X}_j, \quad i = 1, \ldots, m, \quad j = 1, \ldots, n$$

- Problem: estimate m projection matrices $\boldsymbol{P}_i$ and n 3D points $\boldsymbol{X}_j$ from the mn correspondences $\boldsymbol{x}_{ij}$



Is SFM always uniquely solvable?



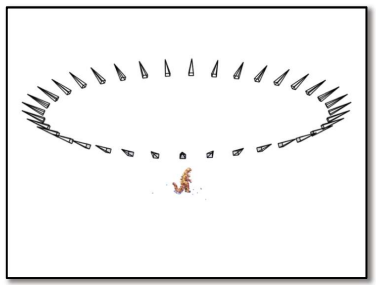
Necker cube

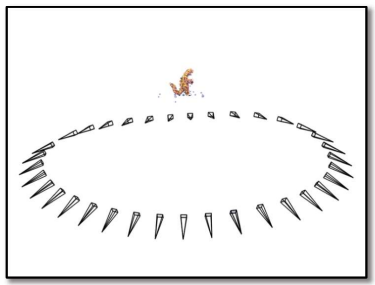
Source: N. Snavely

Is SFM always uniquely solvable?

- Could actually happen in affine structure from motion:







Source: N. Snavely

Structure from motion ambiguity

- If we scale the entire scene by some factor k and, at the same time, scale the camera matrices by the factor of $1/k$, the projections of the scene points remain exactly the same:

$$\boldsymbol{x} \cong \boldsymbol{P}\boldsymbol{X} = \left(\frac{1}{k}\boldsymbol{P}\right)(k\boldsymbol{X})$$

- Without a reference measurement, it is impossible to recover the absolute scale of the scene!
- In general, if we transform the scene using a transformation $\boldsymbol{Q}$ and apply the inverse transformation to the camera matrices, then the image observations do not change:

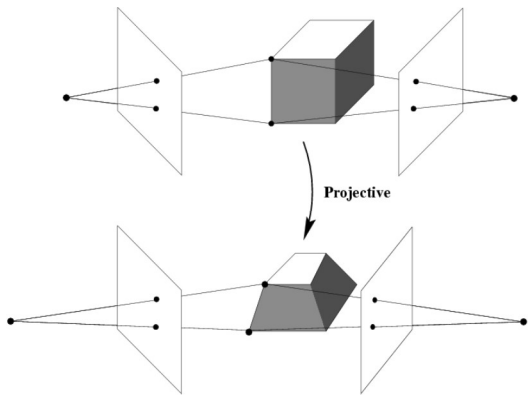
$$\boldsymbol{x} \cong \boldsymbol{P}\boldsymbol{X} = (\boldsymbol{P}\boldsymbol{Q}^{-1})(\boldsymbol{Q}\boldsymbol{X})$$

Projective ambiguity

- With no constraints on the camera calibration matrices or on the scene, we can reconstruct up to a *projective* ambiguity:

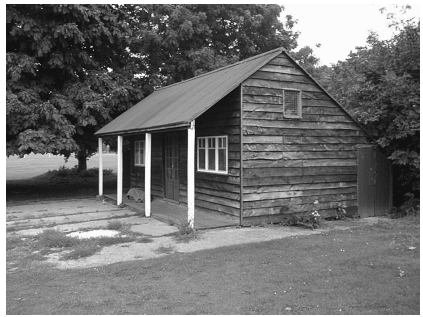
$$\boldsymbol{x} \cong \boldsymbol{PX} = (\boldsymbol{PQ}^{-1})(\boldsymbol{QX})$$

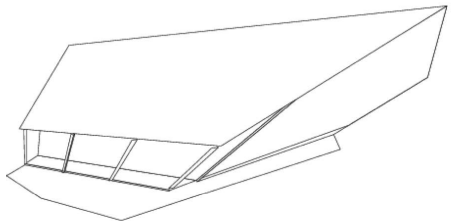
$\boldsymbol{Q}$ is a general full-rank 4×4 matrix

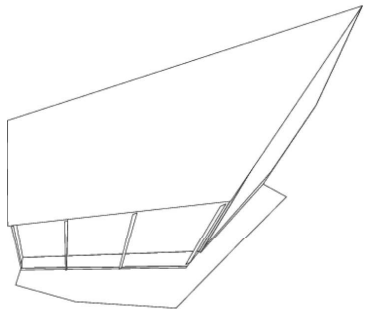


Projective ambiguity









Affine ambiguity

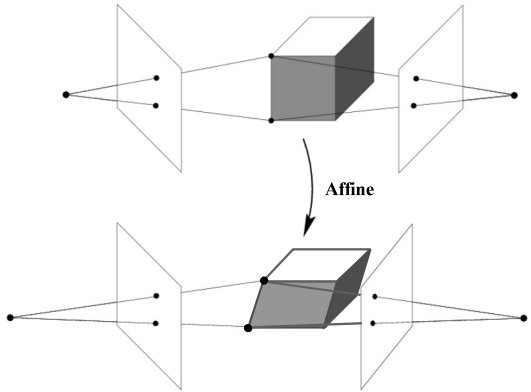
- If we impose parallelism constraints, we can get a reconstruction up to an *affine* ambiguity:

$$\boldsymbol{x} \cong \boldsymbol{PX} = \left(\boldsymbol{PQ}_A^{-1}\right)\left(\boldsymbol{Q}_A\boldsymbol{X}\right)$$

$$\boldsymbol{Q}_A = \begin{bmatrix} \boldsymbol{A} & \boldsymbol{t} \\ \boldsymbol{0}^T & 1 \end{bmatrix}$$

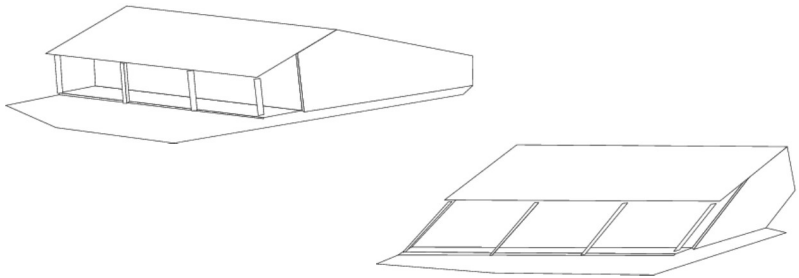
3×3 full-rank matrix

3×1 translation vector



Affine ambiguity





Similarity ambiguity

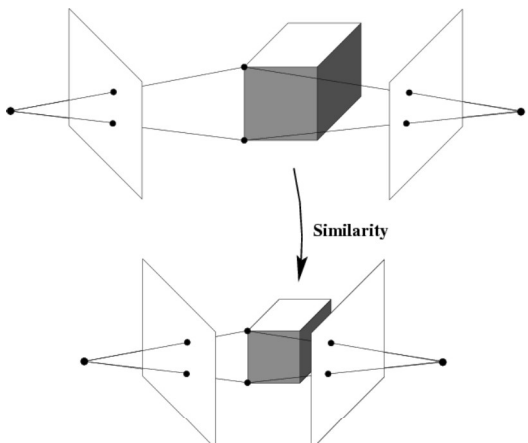
- A reconstruction that obeys orthogonality constraints on camera parameters and/or scene

$$\boldsymbol{x} \cong \boldsymbol{PX} = \left(\boldsymbol{PQ_S^{-1}}\right)\left(\boldsymbol{Q_S X}\right)$$

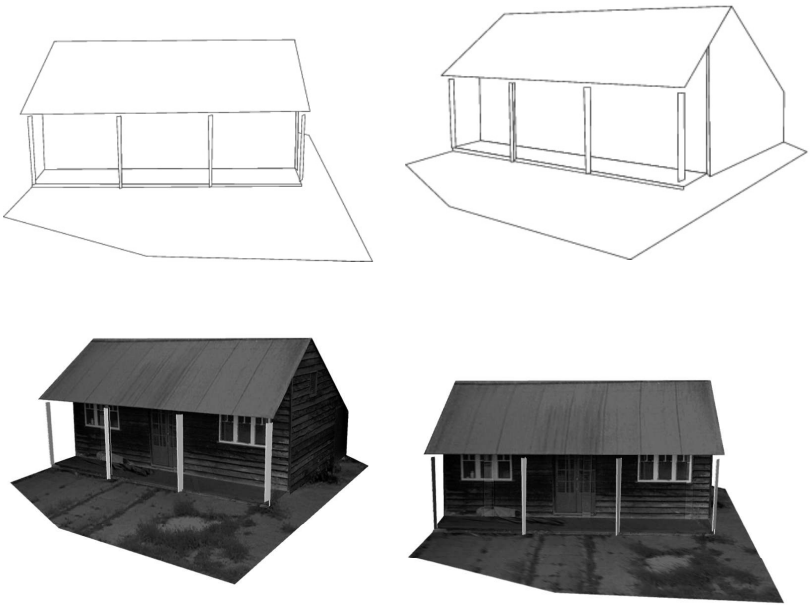
$$\boldsymbol{Q_S} = \begin{bmatrix} s\boldsymbol{R} & \boldsymbol{t} \\ \boldsymbol{0}^T & 1 \end{bmatrix}$$

3×3 rotation matrix

3×1 translation vector



Similarity ambiguity



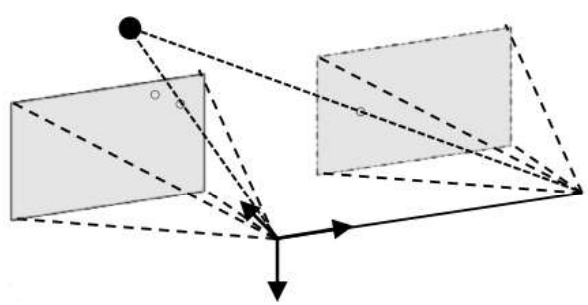
- An agent is moving through the environment and taking images with a rigidly-attached camera system at discrete times k
- In case of a **monocular system**, the set of images taken at times k is denoted by

$$I_{l,0:n} = \{I_0, ..., I_n\}$$

- In case of a **stereo system**, the set of images taken at times k is denoted by

$$I_{l,0:n} = \{I_{l,0}, ..., I_{l,n}\}$$
$$I_{r,0:n} = \{I_{r,0}, ..., I_{r,n}\}$$

In this case, without loss of generality, the coordinate system of the left camera can be used as the origin



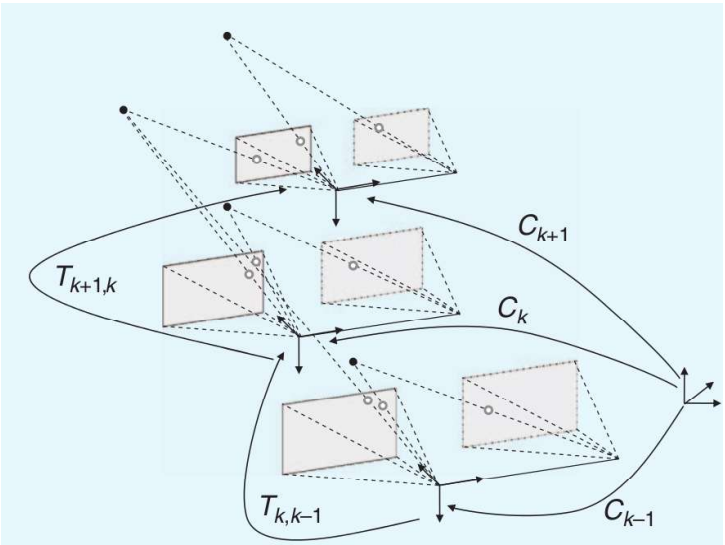
- In case of a **RGB-D camera**, the set of images taken at times k is denoted by

$$I_{0:n} = \{I_0, ..., I_n\}$$
$$D_{0:n} = \{D_0, ..., D_n\}$$



- Two camera positions at adjacent time istants $k-1$ and k are related by the rigid body transformation $T_k = \begin{bmatrix} R_{k-1,k} & t_{k-1,k} \\ 0 & 1 \end{bmatrix}$
- The set $T_{1:n} = \{T_1, ..., T_n\}$ contains all the subsequent motionsk

- The set of camera pose $C_{0:n} = \{C_0, ..., C_n\}$ contains the transformations of the camera w.r.t. the initial coordinate frame at $k = 0$

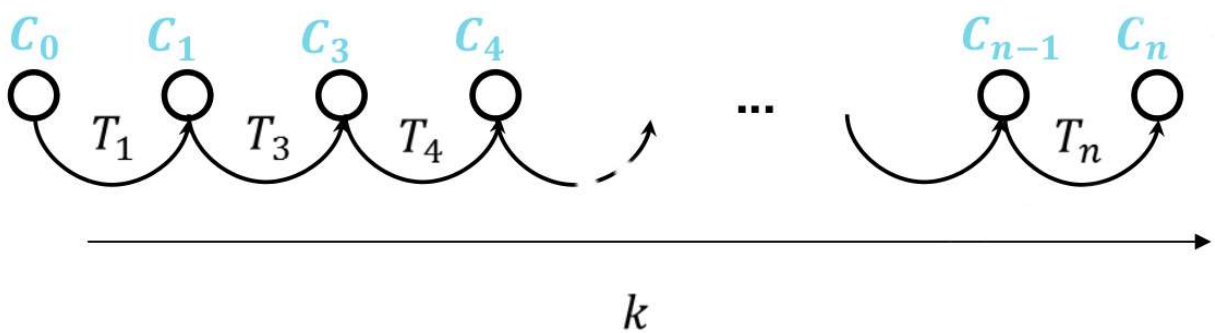


- The current camera pose C_n can be computed by concatenating all the transformations $T_{1:k}$, therefore

$$C_n = C_{n-1} T_n$$

with C_0 being the camera pose at the instant $k = 0$, which can be arbitrarily set by the user

- The main task of VO is to compute the relative transformations T_k from images I_k and I_{k-1} and then to concatenate these transformation to recover the full trajectory $C_{0:n}$ of the camera
- This means that VO recovers the path incrementally, pose after pose



Usual assumptions about the environment

- Sufficient illumination in the environment
- Dominance of static scene over moving objects
- Enough texture to allow apparent motion to be extracted
- Sufficient scene overlap between consecutive frames

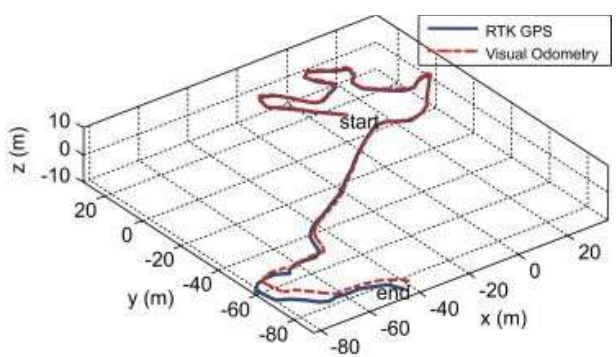
Are these examples OK?





Advantages of Visual odometry

- Contrary to wheel odometry, VO is not affected by wheel slip in uneven terrain or other adverse conditions.
- More accurate trajectory estimates compared to wheel odometry (relative position error 0.1% 2%)
- VO can be used as a complement to wheel odometry
  - GPS
  - inertial measurement units (IMUs)
  - laser odometry
- In GPS-denied environments, such as underwater and aerial, VO has utmost importance

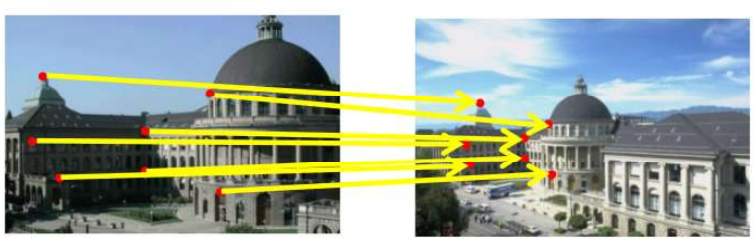


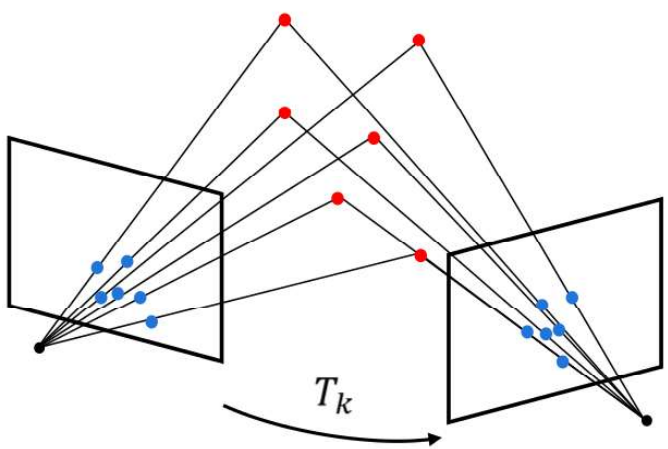


Visual odometry (VO) feature-based

Overview

1. Feature detection
2. Feature matching/tracking
3. Motion estimation
4. Local optimization

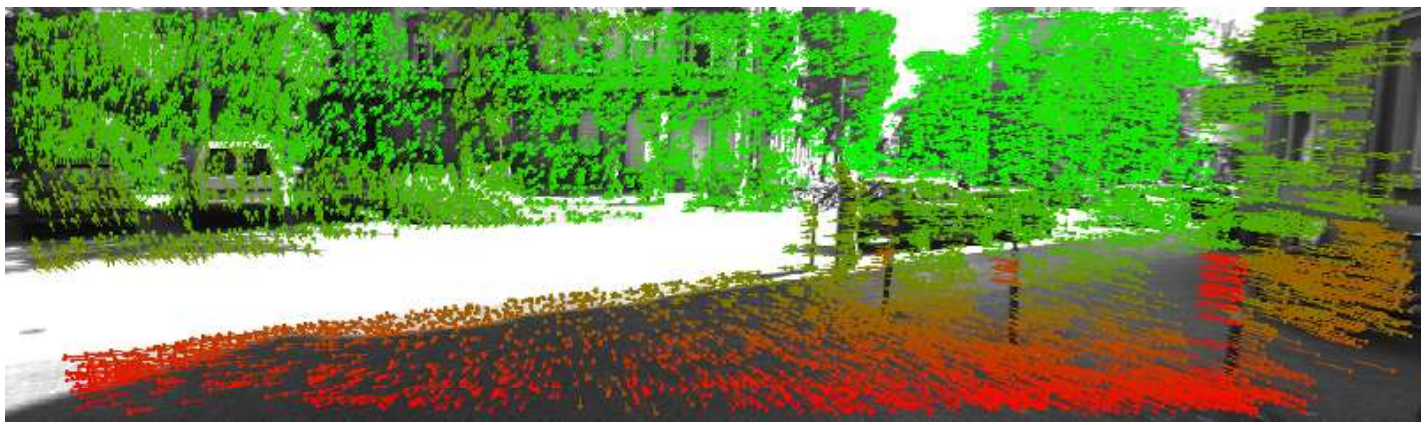






Visual odometry (VO) feature-based
Assumption: camera is well calibrated

1. **Feature detection**:
   Detect a set of features f_k at time k
   (General idea: extract high-contrast areas in the image)



2. **Feature matching/Feature tracking**
   Find correspondences between set of features f_{k-1} , f_k
   - tracking: locally search each feature (e.g. by prediction and correlation)
   - matching: independently detect features in each image and find correspondences on the basis of a similarity metric (exploit descriptors such SURF, SIFT, ORB, etc)

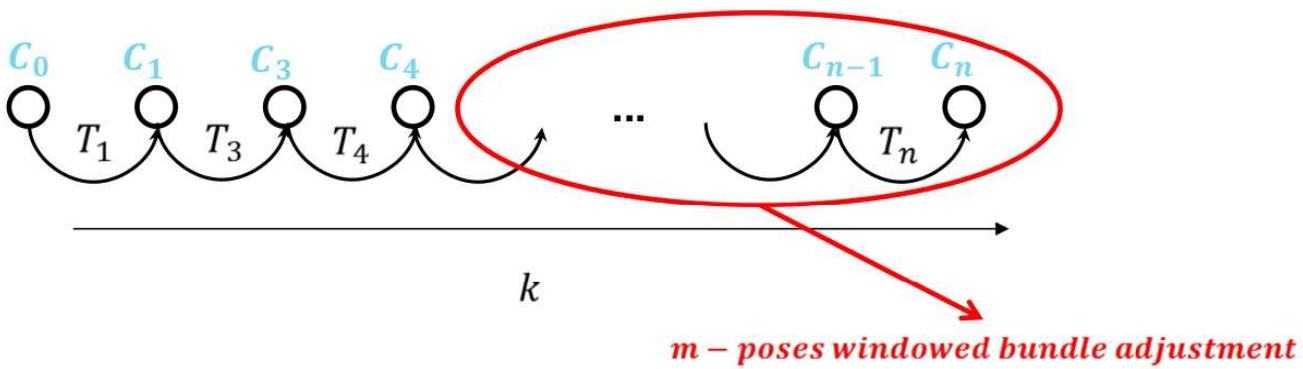
3. **Motion estimation**
   Compute transformation T_k between two images I_{k-1} and I_k from two sets of corresponding features f_{k-1} , f_k.
   Different algorithms depending on available sensor data:

   - *2-D to 2-D*: works on f_{k-1} , f_k specified in 2-D image coords
   - *3-D to 3-D*: works on X_{k-1} , X_k, sets of 3D points corresponding to f_{k-1} , f_k
   - *3-D to 2-D*: works on X_{k-1}, set of 3D points corresponding to f_{k-1}, and on f_k their corresponding 2-D reprojections on the image I_k

Type of correspondences	Monocular	Stereo
2D–2D	X	X
3D–3D		X
3D–2D	X	X

Local optimization

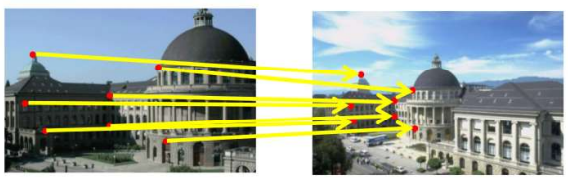
4. An iterative refinement over last m poses can be **optionally** performed after motion estimation to obtain a more accurate estimate of the **local** trajectory

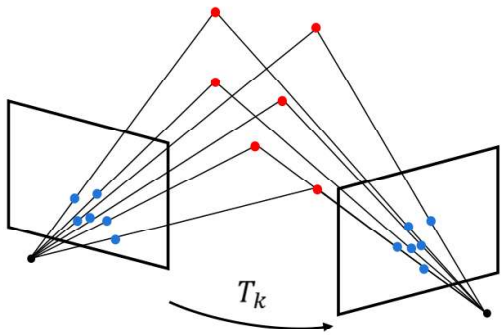


One has to minimize the following image reprojection error

$$T_k = \begin{bmatrix} R_{k-1,k} & t_{k-1,k} \\ 0 & 1 \end{bmatrix} = \arg\min_{X^i, C_k} \sum_{i,k} \| p_k^i - g(X^i, C_k) \|$$

where p_i^k is the i-th image point of the 3D landmark X_i measured in the k-th image and $g(X_i, C_k)$ is its image reprojection according to the current camera pose C_k

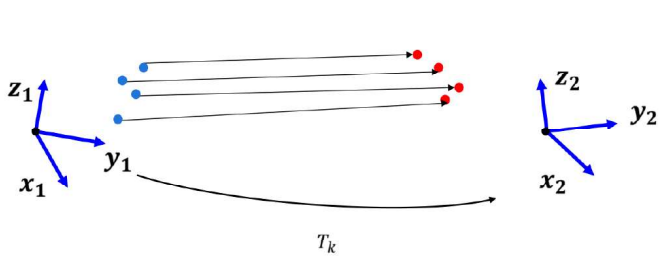


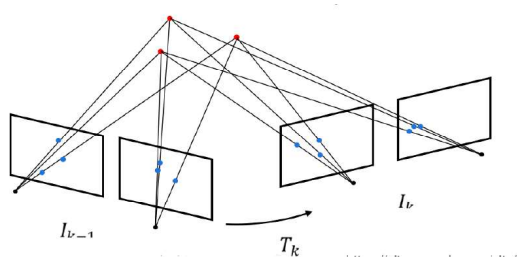


VO from 2-D to 2-D (feature-based)

1. Capture new frame I_k
2. Extract and match features between I_{k-1} and I_k
3. Compute essential matrix for image pair I_{k-1} , I_k
4. Decompose essential matrix into R_k and t_k , and form T_k
5. Compute relative scale and rescale t_k accordingly
6. Concatenate transformation by computing $C_k = C_{k-1}T_k$
7. Repeat from 1).

NOTE: The minimal-case solution involves 5-point correspondences

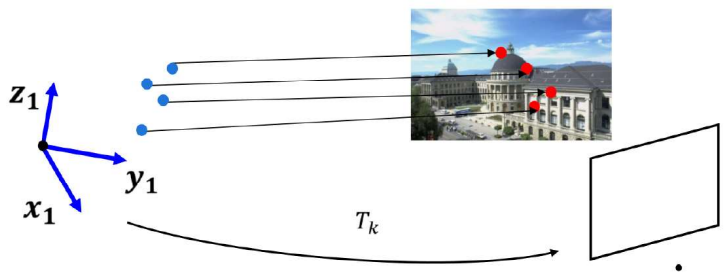




VO from 3-D to 3-D (feature-based)

1. Capture two stereo image pairs $I_{l,k-1}$,$I_{r,k-1}$ and $I_{l,k}$,$I_{r,k}$
2. Extract and match features between $I_{l,k-1}$,$I_{l,k}$
3. Triangulate matched features for each stereo pair. Hence:
   $I_{l,k-1}$, $I_{r,k-1} \Rightarrow X_{k-1}$
   $I_{l,k}$, $I_{r,k} \Rightarrow X_k$
4. Compute T_k from 3-D features X_{k-1} and X_k
5. Concatenate transformation by computing $C_k = C_{k-1} T_k$
6. Repeat from 1).

NOTE: The minimal-case solution involves 3 non-collinear correspondences

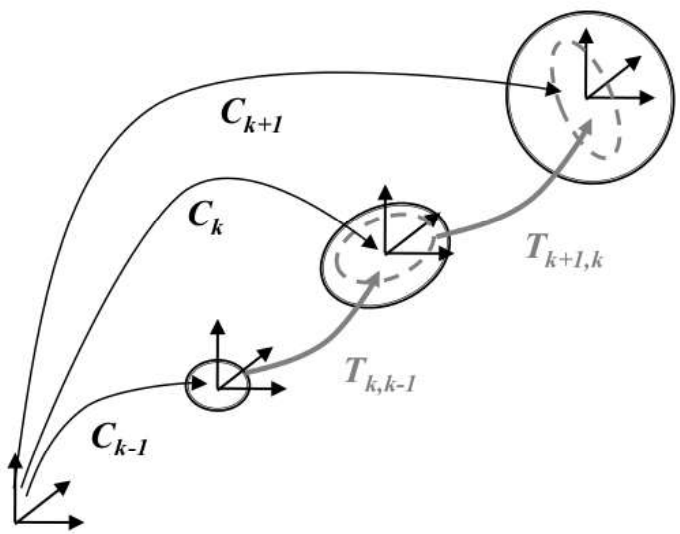


VO from 3-D to 2-D (feature-based)

1. Do only once:
   - 1.1 Capture two frames I_{k-2} , I_{k-1}
   - 1.2 Extract and match features between them
   - 1.3 Triangulate features from I_{k-2} , I_{k-1} and get X_{k-1}
2. Do at each iteration:
   - 2.1 Capture new frame I_k
   - 2.2 Extract features and match with previous frame I_{k-1}
   - 2.3 Compute camera pose (PnP) from 3-D-to-2-D matches (between f_k and X_{k-1})
   - 2.4 Triangulate all new features between I_{k-1} and I_k and get X_k
   - 2.5 Iterate from 2.1

VO drift

1. The errors introduced by each new frame-to-frame motion accumulate over time
2. This generates a drift of the estimated trajectory from the real one



NOTE: the uncertainty of the camera pose at C_k is a combination of the uncertainty at C_{k-1} (black solid ellipse) and the uncertainty of the transformation $T_{k,k-1}$ (gray dashed ellipse)

VO or SFM

1. SFM is more general than VO and tackles the problem of 3D reconstruction of both the structure and camera poses from unordered image sets
2. The final structure and camera poses are typically refined with an offline optimization (i.e., bundle adjustment), whose computation time grows with the number of images

video SFM

VO or SFM

- VO is a particular case of SFM
- VO focuses on estimating the 3D motion of the camera sequentially (as a new frame arrives) and in **real time**.
- Bundle adjustment can be used (but its optional) to refine the local estimate of the trajectory

Pose from point correspondences

Minimise ***geometric error***

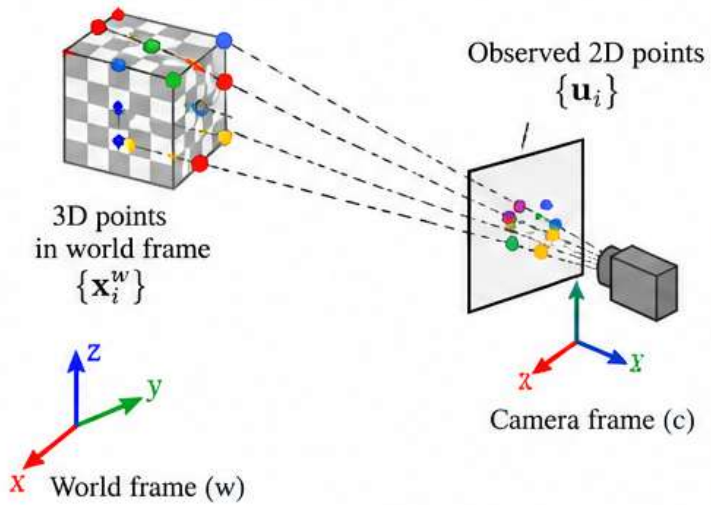
$$\mathbf{T}_{wc}^{*} = \underset{\mathbf{T}_{wc}}{\operatorname{argmin}} \sum_{i} \left\| \pi\left(\mathbf{T}_{wc}^{-1} \cdot \mathbf{x}_i^w\right) - \mathbf{u}_i \right\|^2$$

- $\mathbf{T}_{wc}^{*}$: Optimal camera pose (world → camera)
- $\mathbf{T}_{wc}$: Optimise over camera pose $\mathbf{T}_{wc}$
- $\sum_i$: Sum over all point correspondences
- $\pi\left(\mathbf{T}_{wc}^{-1} \cdot \mathbf{x}_i^w\right)$: Project 3D world point to image using pose
- $\mathbf{u}_i$: Observed 2D image point
- $\left\| \cdot \right\|^2$: Squared reprojection (geometric) error

$$\pi([X, Y, Z]^T) = \left[\frac{f_x X}{Z} + c_x, \frac{f_y Y}{Z} + c_y\right]^T$$

1. Setup: 3D–2D point correspondences

We have N 3D points in world coordinates $\mathbf{x}_i^w$ and their observed 2D image points $\mathbf{u}_i$ in an image of a camera with known intrinsics $\mathbf{K}$.



2. Project 3D point using a pose

Transform a 3D world point to the camera frame using the inverse pose $\mathbf{T}_{wc}^{-1}$ and project to 2D:

$$\mathbf{X}_i^c = \mathbf{T}_{wc}^{-1} \mathbf{x}_i^w = \mathbf{R}^T(\mathbf{x}_i^w - \mathbf{t})$$

$$\hat{\mathbf{u}}_i = \pi(\mathbf{X}_i^c)$$

$$\pi([X, Y, Z]^T) = \left[\frac{f_x X}{Z} + c_x, \frac{f_y Y}{Z} + c_y\right]^T$$

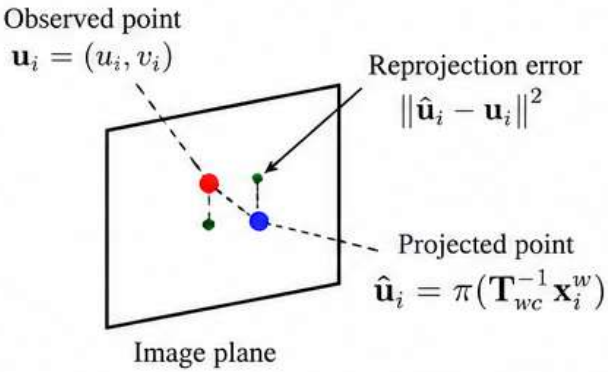
where $\mathbf{T}_{wc} = [\mathbf{R} \mid \mathbf{t}]$ is the world → camera transformation (rotation $\mathbf{R}$ and translation $\mathbf{t}$),

and $\mathbf{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$ contains camera intrinsics.

3. Minimise geometric (reprojection) error

Compare the projected point $\hat{\mathbf{u}}_i$ with the observed image point $\mathbf{u}_i$ and minimise the squared Euclidean distance (reprojection error) over all points:

$$\mathbf{T}_{wc}^{*} = \underset{\mathbf{T}_{wc}}{\operatorname{argmin}} \sum_{i} \left\| \pi\left(\mathbf{T}_{wc}^{-1} \cdot \mathbf{x}_i^w\right) - \mathbf{u}_i \right\|^2$$



Stereo Visual SLAM: Using Projection and Reprojection Error

Estimate camera poses and a 3D map by projecting 3D landmarks into images and minimizing the reprojection error over time.

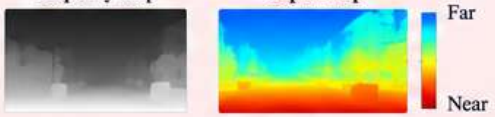
1. At time t: Capture stereo images and compute depth

Left image (t) | Right image (t)



Stereo matching → disparity → depth

Disparity map | Depth map

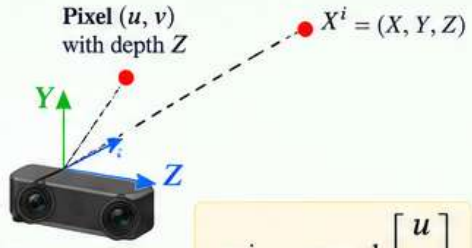


$$Z = \frac{fB}{d}$$

d: disparity f: focal length B: baseline

2. Create 3D landmarks from depth

For each matched feature, back-project to a 3D point in the current camera frame.

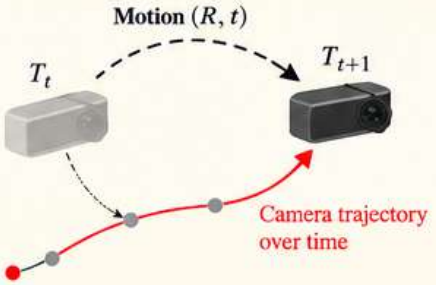


$$X_t^i = ZK^{-1}\begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$$

We now have 3D landmarks X_t^i in the camera coordinate system.

3. Robot moves to time $t + 1$

The robot moves and the camera pose changes from T_t to T_{t+1} (rotation R and translation t).



4. Project old 3D landmarks to the new image (prediction)

Transform the 3D landmark using the estimated motion and project into the new image to get the predicted pixel location.

$$X_{t+1}^{i,\text{pred}} = RX_t^i + t$$
$$\hat{u}_{t+1}^i = \pi\left(X_{t+1}^{i,\text{pred}}\right)$$

Right/left image at $t + 1$



5. Observe the same landmark at $t + 1$

Detect and match the same feature in the new stereo pair and also compute its current depth from stereo (optional).



Stereo gives a new depth (optional)

$X_{t+1}^{i,\text{meas}}$ (from current stereo)

6. Reprojection error

Compare the predicted pixel location with the observed pixel location.



Reprojection error (in pixels):

$$e_i = \hat{u}_{t+1}^i - u_{t+1}^i$$

If we also use measured depth, we can have a 3D error: $e_i^{3D} = X_{t+1}^{i,\text{pred}} - X_{t+1}^{i,\text{meas}}$

7. Update camera pose

Adjust the camera pose (R, t) to minimize the reprojection error over many landmarks (e.g., PnP or nonlinear optimization).

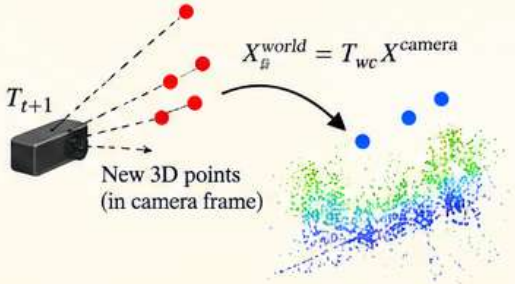
$$T_{t+1}^* = \underset{T}{\text{argmin}} \sum_i \left\| \pi\left(TX_t^i\right) - u_{t+1}^i \right\|^2$$



Many landmarks are used for a robust pose estimate.

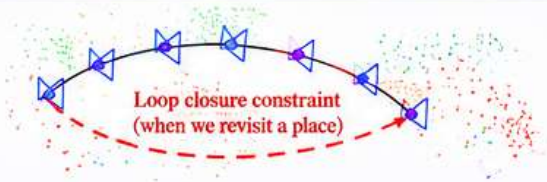
8. Update the map

Triangulate new landmarks from the current stereo pair and transform them to the world coordinate system using the estimated pose.



Add new points to the global map

9. Over many frames: Bundle Adjustment and Loop Closure



Jointly optimize all camera poses and 3D landmarks to minimize the total reprojection error:

$$\min_{\{T_i\},\{X_j\}} \sum_{i,j} \left\| \pi\left(T_i X_j\right) - u_{ij} \right\|^2$$

- Uses all observations from all frames
- Includes loop closure constraints
- Reduces accumulated drift
- Produces a consistent camera trajectory and a global 3D map

Before loop closing | After loop closing

